

Automated E-commerce Cart Recovery System

A scheduled n8n workflow that detects abandoned shopping carts in PostgreSQL and automatically sends personalized recovery emails — built, tested, and debugged end-to-end.

n8n

PostgreSQL

SQL

SMTP

HTML Email

Workflow Automation

1 Project Overview

This project is a scheduled n8n workflow that monitors a `abandoned_carts` table in PostgreSQL, identifies carts marked as abandoned, waits a set period, and automatically emails the customer a personalized reminder with their product and cart value. Tested end-to-end with three sample carts, confirmed delivered to a live inbox.

2 Problem

Shoppers frequently add items to a cart and leave without completing checkout. Without a follow-up, that sale is simply lost. Manually tracking and emailing every abandoned cart doesn't scale, so most of that recoverable revenue goes untouched.

3 Solution

- 1 Schedule Trigger** — starts the workflow on a recurring interval.
- 2 Select rows from a table** — reads all rows from the `abandoned_carts` table in PostgreSQL.
- 3 Check Cart Status (IF)** — routes carts where `status = "ABANDONED"` into the recovery path.
- 4 Wait 1 Hour** — delays before sending, giving the customer time to return on their own.
- 5 Send Email** — a personalized HTML reminder with the product name and cart value, sent to the customer's actual email via a dynamic expression.

6 Update Cart Status — intended to mark the cart as handled so it isn't emailed again.

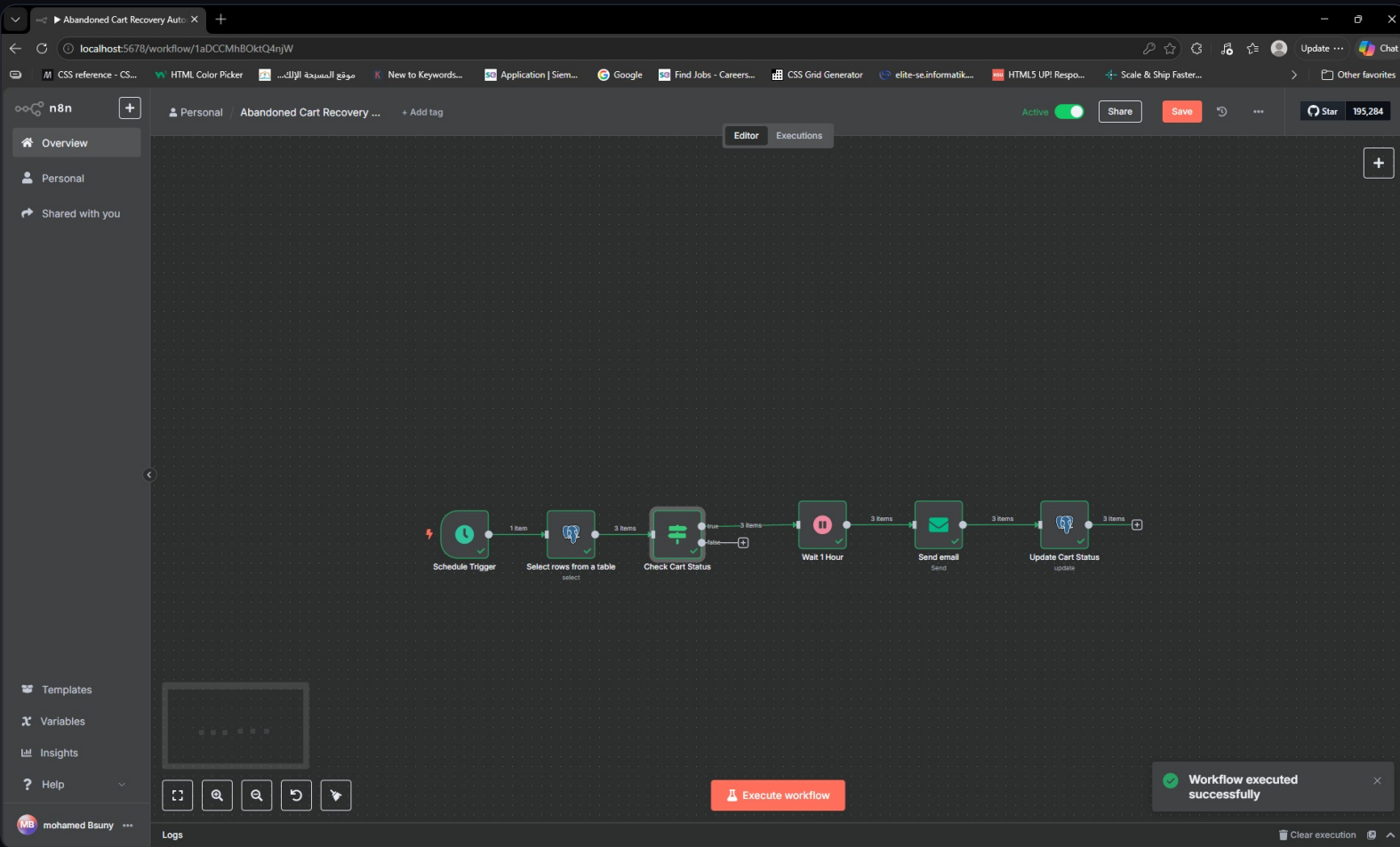
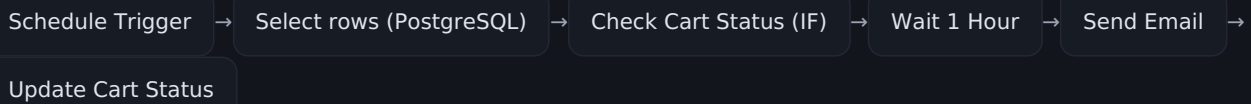


Figure 1 — Full workflow: Schedule Trigger → Select rows from a table → Check Cart Status → Wait 1 Hour → Send email → Update Cart Status. All nodes executed successfully.

4 Architecture



5 Database Design

COLUMN	TYPE	NOTES
id	integer (PK)	Identifier
customer_name	varchar	Customer name

email	varchar	Recipient of the reminder email
product_name	varchar	Item left in the cart
cart_value	numeric	Cart total
status	varchar	e.g. "ABANDONED" — intended to change to "REMINDER_SENT" after the email goes out
created_at	timestamp	Row creation time

TEST DATA NOTE

All three sample rows currently share the developer's own email address rather than distinct customer addresses. The `{{ $json.email }}` expression itself is implemented correctly (see Figure 3) — this is purely a seed-data limitation, worth re-testing with real distinct addresses before client use.

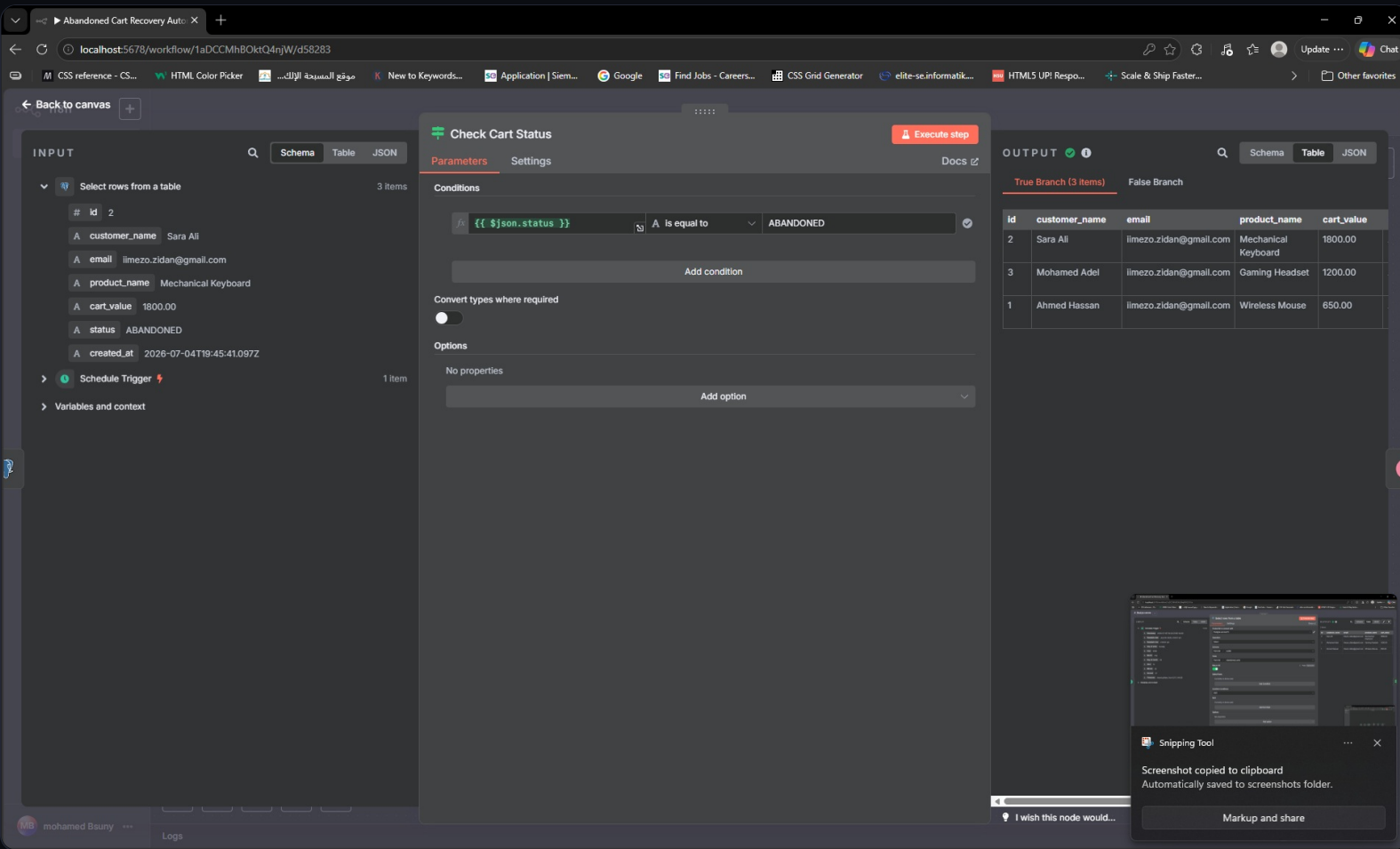


Figure 2 — The IF node correctly routes all three ABANDONED test carts into the True branch.

The screenshot displays the n8n workflow editor interface. On the left, the 'INPUT' section shows a 'Wait 1 Hour' node followed by a 'Check Cart Status' node, which triggers a 'Schedule Trigger' node. The main workspace shows the 'Send email' node configuration. The 'Parameters' tab is active, showing the 'SMTP account 2' credential, 'Send' operation, and 'From Email' set to 'limezo.zidan@gmail.com'. The 'To Email' field uses a dynamic expression: `{{ $json.email }}`. The 'Subject' is 'You left something in your cart', and the 'Email Format' is 'HTML'. The HTML content is a personalized message:

```

<h2>Hi {{ $json.customer_name }},</h2>
<p>You recently left <b>{{ $json.product_name }}</b> in your shopping cart.</p>
<p>Total Cart Value: <b>{{ $json.cart_value }}</b>
<h2>Hi Sara Ali,</h2> <p>You recently left <b>Mechanical Keyboard

```

The 'Options' section has 'Append n8n Attribution' disabled. On the right, the 'OUTPUT' section shows three items, all with an 'accepted' status. The output table is as follows:

accepted	rejected	ehlo	envelopeTime	message
0 : limezo.zidan@gmail.com	[empty array]	0 : SIZE 35882577 1: 8BITMIME 2: AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH 3: ENHANCEDSTATUSCODES 4: PIPELINING 5: CHUNKING 6: SMTPUTF8	357	687
0 : limezo.zidan@gmail.com	[empty array]	0 : SIZE 35882577 1: 8BITMIME 2: AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH 3: ENHANCEDSTATUSCODES 4: PIPELINING 5: CHUNKING 6: SMTPUTF8	330	638
0 : limezo.zidan@gmail.com	[empty array]	0 : SIZE 35882577 1: 8BITMIME 2: AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH 3: ENHANCEDSTATUSCODES 4: PIPELINING 5: CHUNKING 6: SMTPUTF8	346	752

Figure 3 — Send Email node: To Email correctly uses a dynamic `{{ $json.email }}` expression, and all three sends were accepted.

6 Features (Implemented)

- Scheduled, unattended execution
- PostgreSQL read of live cart data
- Conditional routing by cart status
- Configurable delay before sending
- Personalized HTML email per customer
- Dynamic recipient targeting (no hardcoded address)
- Database write-back step (see Known Issues)

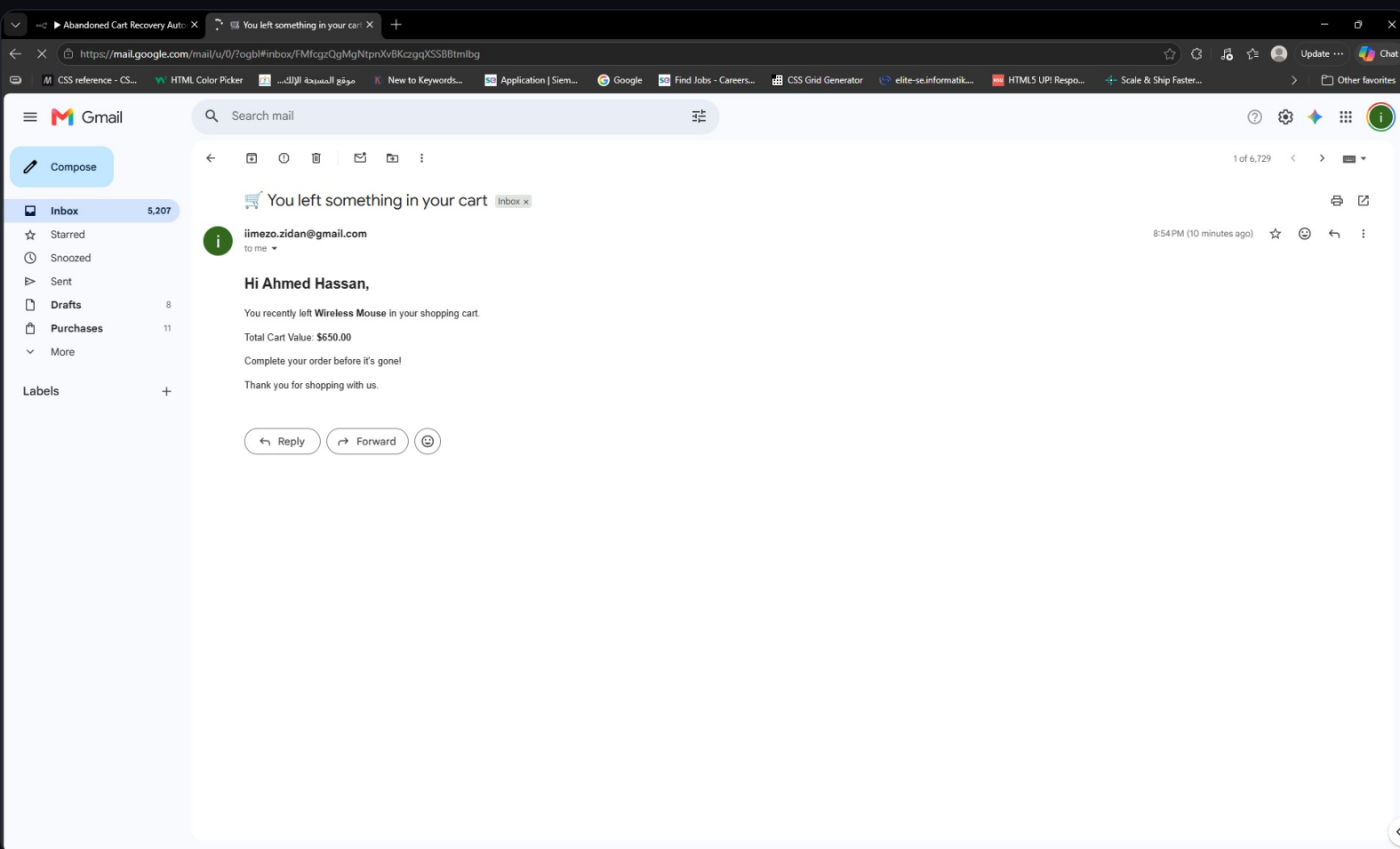


Figure 4 — Delivered email: correct product name and cart value for Ahmed Hassan's Wireless Mouse.

7 Business Value

Recovers lost sales

Reaches customers who would otherwise never come back to finish checkout.

Zero manual follow-up

No one has to track or send these reminders by hand.

Consistent messaging

Every customer gets the same accurate, on-brand reminder.

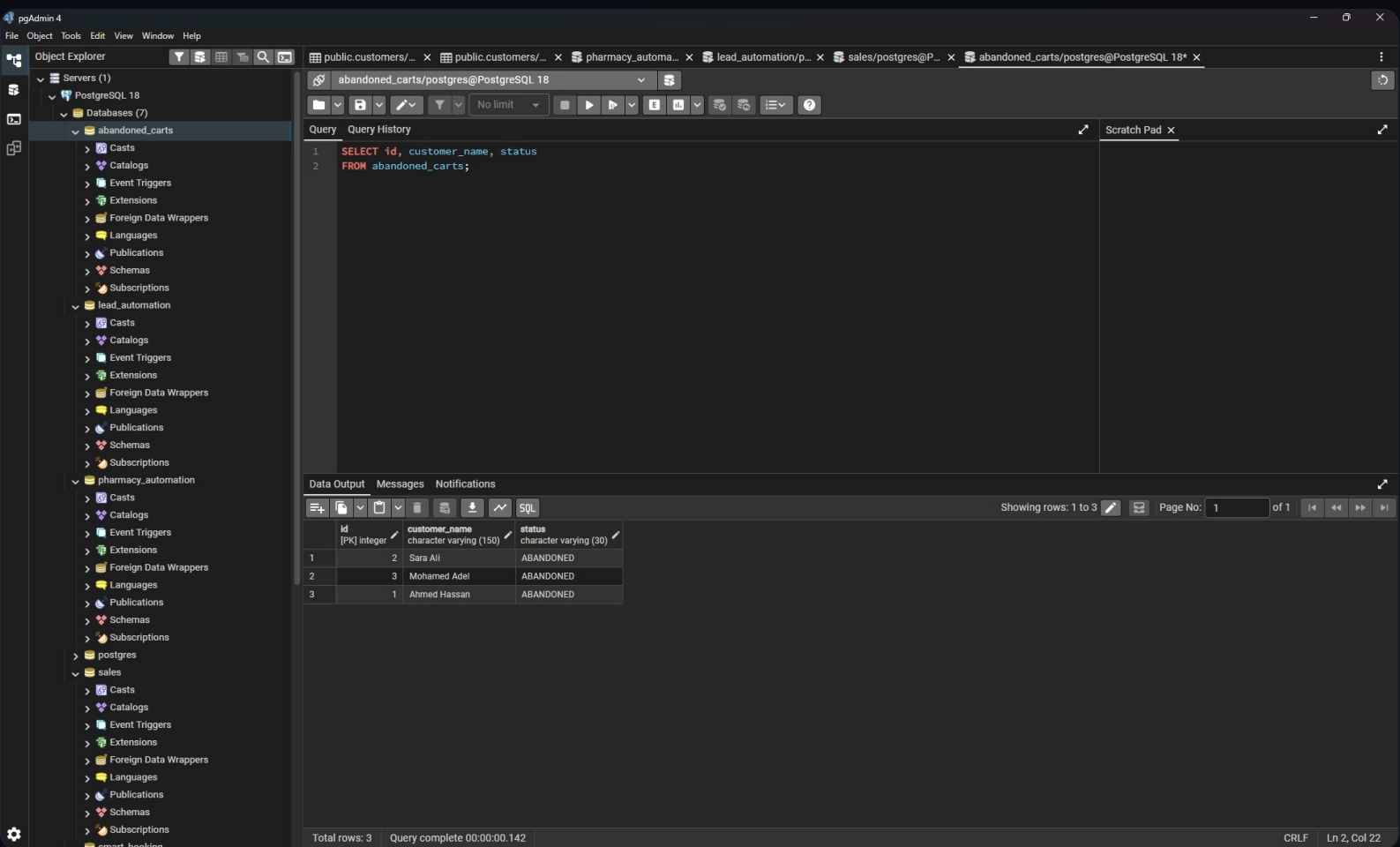


Figure 5 — After a successful run, all three carts are still marked "ABANDONED" in PostgreSQL.

8 Known Issues & Future Improvements

✓ **Fixed & verified** The Update Cart Status node now correctly writes `status → "REMINDER_SENT"` after each email is sent. A follow-up database query confirms all three test carts moved from "ABANDONED" to "REMINDER_SENT," so the IF node will correctly skip them on future scheduled runs instead of re-sending duplicate reminders.

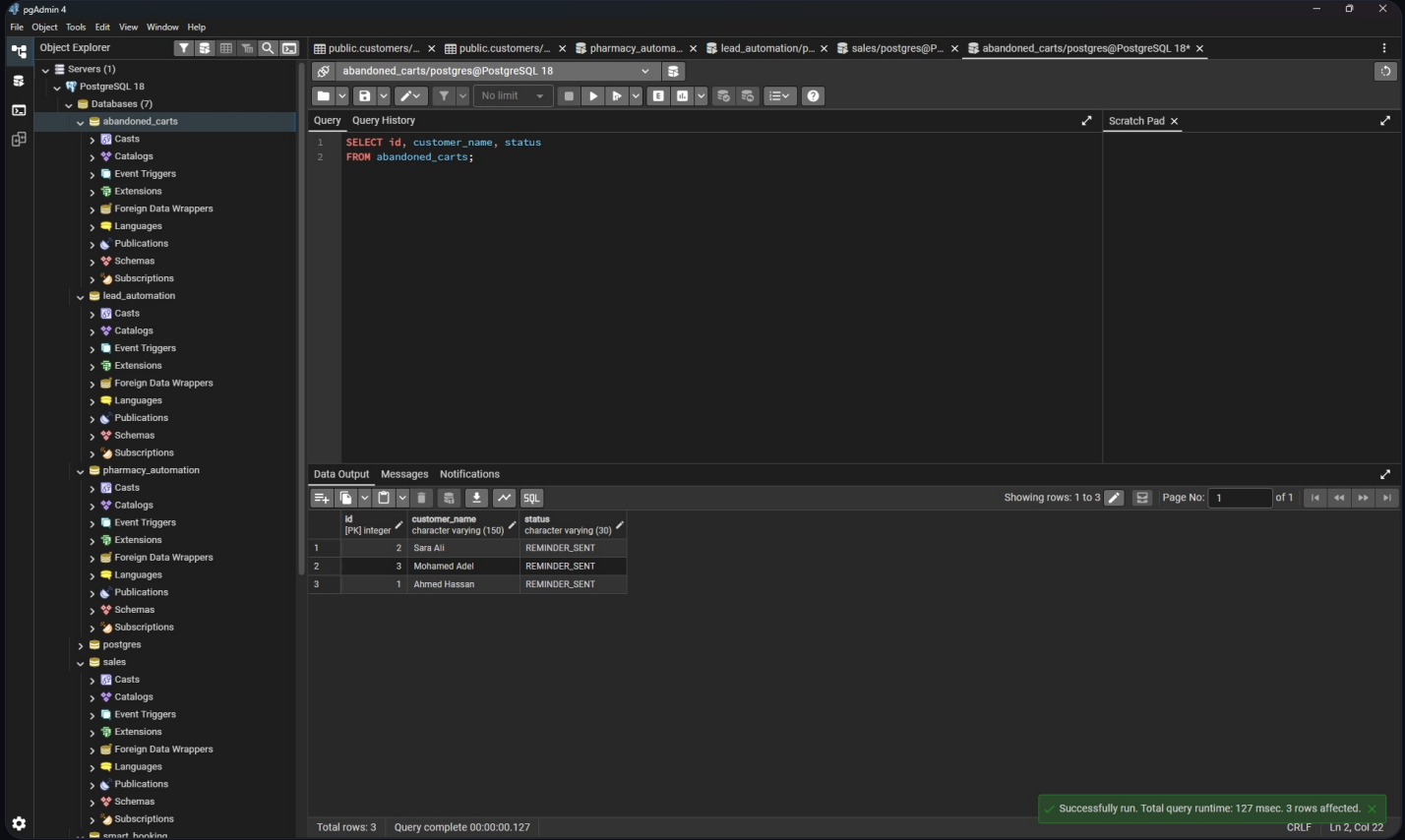


Figure 6 — Verification query after the fix: all three carts now show status "REMINDER_SENT."

ALSO WORTH CHECKING

Only the "abandoned" path (IF → True branch) has been exercised in testing; there's no confirmed test of a non-abandoned cart correctly skipping the email via the False branch. The exact schedule interval also wasn't visible in the screenshots reviewed — worth double-checking it's set to a sensible frequency (e.g. hourly, not every minute) before going live.

9 Skills Demonstrated

Scheduled workflow automation (n8n)

Conditional (IF) branching logic

PostgreSQL read/update operations

Dynamic, per-recipient email personalization

End-to-end QA — caught a real state-management bug by cross-checking the database after execution

10 Portfolio-Ready Description

Built an automated cart-recovery workflow in n8n that detects abandoned carts in PostgreSQL on a schedule and sends personalized HTML reminder emails with correctly dynamic recipient targeting. QA testing — cross-

checking the database after each run — caught a real bug where cart status was never updated after sending; diagnosed the root cause, fixed it, and verified with a follow-up database query that customers are no longer re-emailed once handled.

Case study generated from live n8n execution logs, PostgreSQL query results, and delivered email screenshots.